

Ayudantía 6

Contenedores parte 2

Programación Avanzada | Sección: 03 | CIT 1010
Universidad Diego Portales

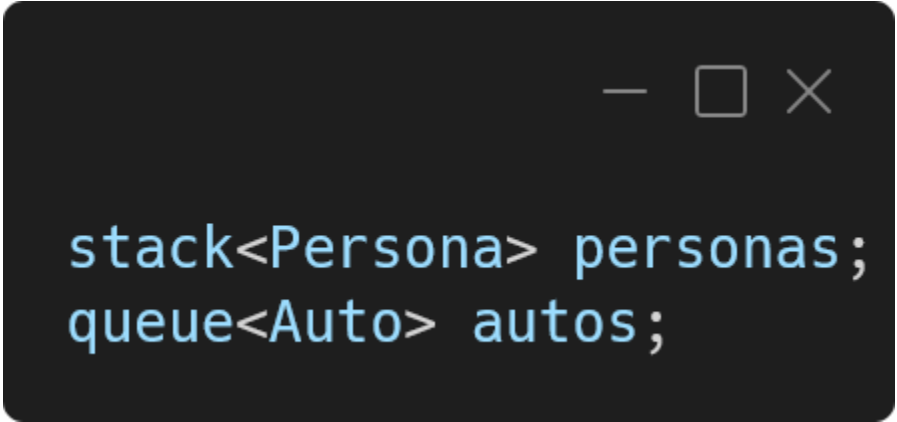
Contacto: isaias.vasquez1@mail.udp.cl **Discord:** _iszzza
Github: github.com/Isza285/Programacion-Avanzada-2026-01

Sección 1

Objetos en contenedores

Objetos en contenedores

Los contenedores pueden almacenar objetos creados desde clases.

A dark-themed code editor window with standard window controls (minimize, maximize, close) in the top right corner. It contains two lines of C++ code: `stack<Persona> personas;` and `queue<Auto> autos;`.

```
stack<Persona> personas;  
queue<Auto> autos;
```

Acceso a objetos dentro del contenedor

Utilizando métodos como `top()`, `front()` y `back()` nos retornaran los objetos.

Importante: Se utiliza `.` porque el contenedor almacena objetos y no punteros.

```
//Ejemplo con stack
cout << personas.top().nombre << endl;
//Ejemplo con queue
cout << clientes.front().nombre << endl;
```

Recorrer contenedores de objetos

Es posible recorrer contenedores utilizando ciclos while.

```
//Ejemplo con stack
while(!personas.empty()){
    cout << personas.top().nombre << endl;
    personas.pop();
}
```

```
//Ejemplo con queue
while(!clientes.empty()){
    cout << clientes.front().nombre << endl;
    clientes.pop();
}
```

Sección 2

Contenedores como atributos

Contenedores como atributos

Las clases pueden tener contenedores como stack y queue dentro de sus atributos.

```
class Curso{  
public:  
    stack<int> notas;  
};
```

```
Curso c;  
c.notas.push(70);  
c.notas.push(55);  
  
cout << c.notas.top() << endl;
```

Constructores

Los stack y queues se pueden utilizar directamente dentro de un constructor.

Ejemplo en main:

```
Banco b;  
  
cout << b.clientes.front();  
//Output: Ana
```

```
class Banco{  
public:  
  
    queue<string> clientes;  
  
    Banco( ){  
  
        clientes.push("Ana");  
        clientes.push("Luis");  
  
    }  
  
};
```

Importante

No es necesario inicializar manualmente una queue o stack.

Esto debide a que el contenedor ya tiene un constructor propio.

En cambio los arreglos necesitan ciclos o listas de inicialización.



```
//Redundante.  
Banco() : clientes() {}
```

Ejemplo

Simulación de un banco:

<https://onlinegdb.com/EWiHn8nj>

Sección 3

Ejercicios

Ejercicio 1

Crear una clase biblioteca que tenga como atributo `stack<string>` llamado `libros`.

- Crear un método `agregarLibro()` para insertar libros.
- Crear un método `mostrarLibros()` para mostrar todos los libros dentro del `stack`.
- Añadir al menos 3 libros en el `main` y utilizar el método `mostrarLibros()`.

Ejercicio 2

Crear una clase Restaurante que tenga como atributo una `queue<string>` llamada pedidos.

- Crear un método `agregarPedido()`. Que añade un pedido.
- Crear un método `prepararPedido()`. Que prepara el primer pedido que esta en la cola y lo elimina con `pop()`.
- Agregar 3 pedidos en el main y preparar 1 pedido.

Ejercicio 3

Crear una clase Producto que contiene nombre y precio como atributos.

- Crear un stack<Producto>.
- Agregar 3 productos.
- Mostrar el nombre y precio de cada producto con top().

Ejercicio 4

Crear una clase Supermercado que contiene como atributo una `queue<Cliente>`.

Cliente contiene lo siguiente:

- Nombre.

Se debe añadir a la clase Supermercado:

- Metodo `agregarCliente()`.
- Método `atenderCliente()` y mostrar el nombre del cliente.

En el main añadir al menos 3 clientes y atenderlos a todos hasta que la cola quede vacía.